
SPOTPY como herramienta de experimentación, sensibilidad y calibración para modelos hidrológicos:

laboratorio reproducible orientado a SWAT+ IGP

Proyecto PP00089 - Instituto Geofísico del Perú

Autor editable: Jhon Gesell Villanueva Portella

Versión del laboratorio: SPOTPY 1.6.7

9 de agosto de 2026

Resumen ejecutivo

Este manual documenta un laboratorio ejecutado y verificable, no sólo una propuesta. Explica qué recibe SPOTPY, cómo envuelve un modelo, qué persiste y cómo se interpretan exploración, sensibilidad, calibración, multiobjetivo e incertidumbre. MC, LHS, FAST, SCE-UA y DDS están probados; las capacidades avanzadas se clasifican con cautela. La conexión con SWAT+ se presenta como arquitectura futura: no se afirma una integración real sin ejecutable, proyecto, parámetros, unidades y parser verificados.

Índice general

Resumen ejecutivo	1
1. Introducción	5
2. Arquitectura de SPOTPY	6
3. Entorno verificado	7
4. Entradas, procesos y salidas	8
5. Rosenbrock: contrato mínimo	11
6. Modelo hidrológico didáctico	12
7. Funciones objetivo y métricas	13
8. Sensibilidad FAST y eFAST	14
9. Calibración: SCE-UA y DDS	15
10. Control de un modelo externo	16
11. Arquitectura futura SWAT+ IGP	17
12. Capacidades avanzadas evaluadas	19
12.1. Firmas hidrológicas	19
12.2. NSGA-II	19
12.3. DREAM y posterior	19
12.4. Bases	19
13. Reproducibilidad	20
14. Conclusiones	21

Índice de figuras

2.1. Flujo ejecutado por el setup y el sampler.	6
11.1. Arquitectura propuesta; no representa una ejecución SWAT+ ya implementada. .	17

Índice de cuadros

3.1. Versiones ejecutadas.	7
4.1. Tabla maestra de entradas, procesos y salidas.	9
6.1. Parámetros didácticos; no son parámetros SWAT+.	12

Capítulo 1

Introducción

SPOTPY es una biblioteca Python para estimar, explorar y analizar parámetros de modelos mediante algoritmos intercambiables [6]. Resuelve el problema operativo de conectar un modelo arbitrario con muestreo, optimización, sensibilidad o incertidumbre sin reescribir el algoritmo para cada modelo.

No limpia precipitación, corrige temperatura, procesa DEM, delimita cuencas, genera HRU ni construye proyectos SWAT+. Esas tareas pertenecen al preprocesamiento. SPOTPY empieza cuando existe un contrato reproducible que transforma parámetros y forzantes en una simulación comparable con observaciones.

Un parámetro calibrable es una entrada incierta expuesta al sampler mediante nombre, distribución prior y límites. Una iteración es una evaluación de un conjunto: se ejecuta el modelo, se obtiene una serie y se calcula un objetivo. El sampler genera conjuntos, objetivos, simulaciones y, según el método, cadenas o índices de sensibilidad. La calibración busca buen ajuste en un período; la validación comprueba capacidad fuera de ese período sin reajustar. La incertidumbre describe distribuciones plausibles, no sólo un óptimo.

El laboratorio usa un modelo lluvia-caudal didáctico de 30 pasos. Por ello demuestra operatividad, no validez hidrológica para una cuenca peruana.

Capítulo 2

Arquitectura de SPOTPY

Una clase setup implementa cuatro puntos de integración:

`parameters()` declara priors, límites y nombres y entrega candidatos al algoritmo.

`simulation(vector)` aplica el vector al modelo y devuelve la serie simulada.

`evaluation()` entrega observaciones alineadas en longitud y orden.

`objectivefunction(simulation,evaluation)` reduce la comparación a un escalar o vector de objetivos.

El sampler llama repetidamente ese contrato y su base de datos añade prefijos `like*`, `par*`, `simulation_*` y `chain`. El mejor candidato se determina respetando la orientación real: máximo para scores como NSE en DDS; mínimo para la pérdida interna de SCE-UA.

Flujo operativo del laboratorio SPOTPY

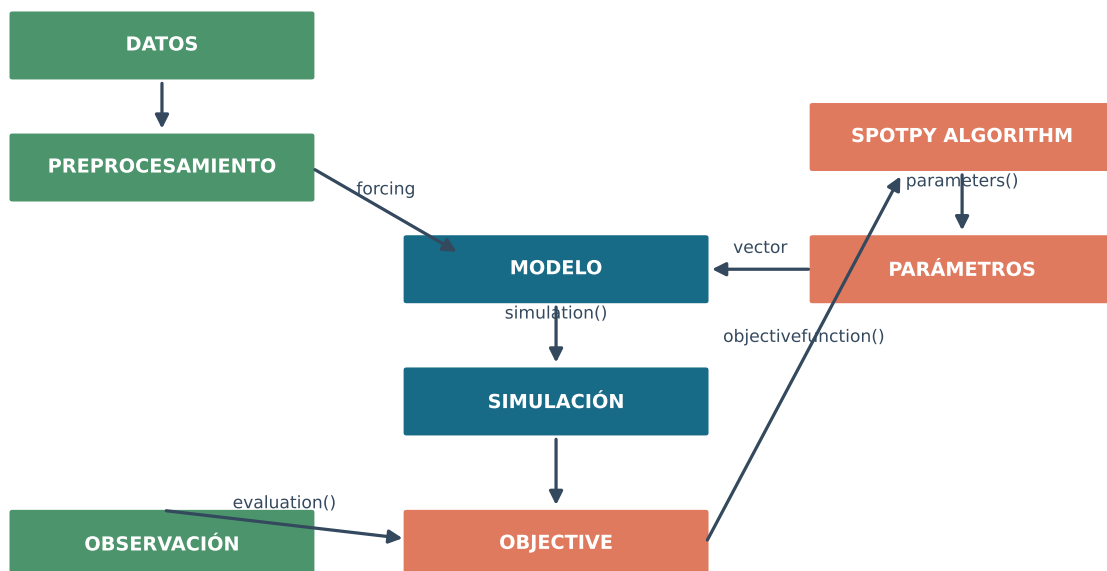


Figura 2.1: Flujo ejecutado por el setup y el sampler.

La figura 2.1 separa el preprocesamiento de SPOTPY. La flecha de retorno representa la decisión del algoritmo sobre el siguiente conjunto.

Capítulo 3

Entorno verificado

Se creó un entorno Conda aislado; no se modificó base ni Python global.

Cuadro 3.1: Versiones ejecutadas.

Componente	Versión
Python	3.12.13
SPOTPY	1.6.7
NumPy	2.5.2
SciPy	1.18.0
Pandas	3.0.5
Matplotlib	3.11.1
pytest	9.1.1

La instalación no incluye mpi4py, PyTables/HDF5, pathos ni multiprocessing. La ejecución validada es secuencial. `environment.yml` y `requirements.txt` fijan las versiones. `pip check` no reporta requisitos rotos.

Capítulo 4

Entradas, procesos y salidas

El input hidrológico concreto se encuentra en `03_hydrology_demo/data/input.csv`. Contiene `date`, `precip_mm`, `pet_mm` y `q_obs`. Las dos variables climáticas están nombradas en mm; el código no define si son acumulados o promedios. La unidad de caudal no está definida: no debe presentarse como m^3/s .

El lector rechaza archivo ausente, columnas faltantes, NaN, fechas inválidas, duplicadas o desordenadas, valores no numéricos y negativos. La tabla completa de campos y validaciones está en `docs/07_input_process_output.md`.

Cuadro 4.1: Tabla maestra de entradas, procesos y salidas.

Etapa	Entrada concreta	Formato	Unidad	Proceso	Salida concreta	Formato	Unidad
Clima	date, precip_mm, pet_mm	CSV	fecha, mm, mm	Validación Pandas	forcing	DataFrame	mismas
Observación	q_obs	CSV float	no definida	validación/alineamiento	evaluation	array	no definida
Modelo	forcing + 5 parámetros	arrays	mixta	balances de reservorios	q_sim	array	no definida
Parámetros	rangos Uniform	objetos	mixta	parameters()	candidato	ParameterSet	mixta
Simulación	candidato + forcing	Python	mixta	simulation()	serie	array	no definida
Evaluación	q_obs	DataFrame	no definida	evaluation()	referencia	array	no definida
Objetivo	sim + obs	arrays	no definida	objectivefunction()	score/loss	float/list	[-], %, o Q
Sampling	setup, runs, seed	CLI/config	n/a	sample()	evaluaciones	CSV/RAM/SQL	mixta
FAST	325 candidatos	diseño FAST	parámetros	Fourier	S1/ST	CSV/PNG	[-]
SCE-UA	120 nominales	sampler	n/a	complejos; minimiza -NSE	calibrado	CSV/JSON/PNG	mixta
DDS	120	sampler	n/a	dimensiones dinámicas; maximiza NSE	calibrado	CSV/JSON/PNG	mixta
Externo	JSON + input CSV	archivos	mixta	subprocess + timeout	output.csv	CSV	no definida
Futuro SWAT+	copia + parámetros verificados	por definir	por definir	swatplus.exe	Q simulada	por verificar	objetivo futuro: m³/s

Cada run conserva configuración, entorno, registro, base SPOTPY, métricas, parámetros, serie inicial/mejor y figuras. Los archivos centrales son `config.json`, `environment.json`, `run.log` y `metrics.json`. Las columnas `simulation_0...29` no incluyen fecha; el índice temporal se recupera del input validado.

Capítulo 5

Rosenbrock: contrato mínimo

Rosenbrock separa la mecánica SPOTPY de la hidrología. Los parámetros son $x \in [-2, 2]$ e $y \in [-1, 3]$ y la pérdida es

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

El óptimo global es $(1, 1)$ con $f = 0$. En 100 muestras MC y semilla 42, el laboratorio obtuvo $f = 0.188121$, $x = 0.763751$, $y = 0.546941$. No alcanzar exactamente el óptimo con un presupuesto corto es esperable. El ejemplo enseña base CSV, selección por mínimo y trazabilidad sin confundir resultados con una simulación física.

Capítulo 6

Modelo hidrológico didáctico

El modelo contiene reservorios de suelo, respuesta rápida y base. Precipitación y PET impulsan 30 pasos diarios. Sus parámetros se listan en Table 6.1.

Cuadro 6.1: Parámetros didácticos; no son parámetros SWAT+.

Parámetro	Mín.	Máx.	Prior	Unidad
runoff_coeff	0.05	0.85	Uniform	[-]
soil_capacity	20	250	Uniform	mm, por consistencia interna
et_factor	0.2	1.5	Uniform	[-]
quick_recession	0.15	0.95	Uniform	fracción/paso
base_recession	0.005	0.25	Uniform	fracción/paso

MC con 50 muestras produjo NSE 0.908542 y KGE 0.898044. Estos valores verifican que el pipeline puede recuperar un buen candidato respecto del conjunto sintético; no validan estructura, unidades ni capacidad predictiva en otra época. Una calibración científica necesita warm-up, separación calibración/validación, incertidumbre observacional y datos suficientemente largos.

Capítulo 7

Funciones objetivo y métricas

Con observado O_i y simulado S_i , NSE es

$$\text{NSE} = 1 - \frac{\sum_i (S_i - O_i)^2}{\sum_i (O_i - \bar{O})^2}.$$

Su ideal es 1 y se maximiza. KGE combina correlación, razón de variabilidad y razón de medias; su ideal también es 1 [5]. RMSE y MAE conservan la unidad desconocida del caudal y se minimizan. PBIAS se expresa en porcentaje; en este laboratorio positivo significa sobreestimación. R^2 mide asociación, no ausencia de sesgo.

SCE-UA está configurado por SPOTPY 1.6.7 para minimizar; por ello recibe $-\text{NSE}$. DDS maximiza y recibe NSE natural. Los JSON vuelven a calcular las métricas con signo hidrológico natural. Esta separación evita elegir accidentalmente el peor candidato.

Una likelihood no es automáticamente una métrica. Para DREAM se usó una likelihood gaussiana con error de medición, no NSE. Su nombre en la API es

```
gaussianLikelihoodMeasErrorOut
```

La validez bayesiana depende de un modelo explícito y defendible del error.

Capítulo 8

Sensibilidad FAST y eFAST

Sensibilidad pregunta cuánto cambia un output al variar inputs; calibración busca valores que ajustan observaciones. FAST asigna frecuencias a parámetros y descompone varianza mediante Fourier [1]. Con 325 evaluaciones, el ranking ST fue: runoff_coeff 0.88951; quick_recession 0.39920; base_recession 0.04214; soil_capacity 0.03988; et_factor 0.02907.

Un ST alto significa influencia dentro de estos rangos, este modelo y esta métrica; no prueba causalidad universal ni identifica el mejor valor. Antes de SWAT+ ayuda a reducir/priorizar parámetros si sus rangos son defendibles.

eFAST extiende el diseño para estimar contribuciones espectrales [7]. SPOTPY exigió 71 corridas para cinco parámetros y devolvió fracciones parciales: 0.3138 para runoff_coeff y 0.1293 para quick_recession, las dos mayores. El mínimo exacto demuestra ejecución, no estabilidad comparativa con FAST.

Capítulo 9

Calibración: SCE-UA y DDS

SCE-UA evoluciona complejos para afrontar óptimos múltiples en modelos lluvia-caudal [3]. Se solicitaron 120 evaluaciones; SPOTPY realizó 225 internas y guardó 88. El mejor interno alcanzó NSE 0.954195, KGE 0.860940, RMSE 0.657595 y PBIAS -8.996 %. El análisis usa el estado interno porque el CSV omitió candidatos descartados por la evolución.

DDS reduce dinámicamente el número de dimensiones perturbadas conforme avanza el presupuesto [8]. Con 120 evaluaciones obtuvo NSE 0.963318, KGE 0.981289, RMSE 0.588474 y PBIAS 0.292 %.

DDS resultó mejor en este caso, semilla y presupuesto; no demuestra superioridad universal. Validación significaría congelar parámetros y evaluar otro período, tarea pendiente por longitud del dataset.

Capítulo 10

Control de un modelo externo

El adaptador ficticio materializa el puente hacia un solver: crea directorio UUID, escribe parámetros, lanza un proceso con `subprocess.run`, fija `cwd/timeout`, captura `stdout/stderr`, verifica return code y archivo, parsea `output.csv` y limpia o preserva.

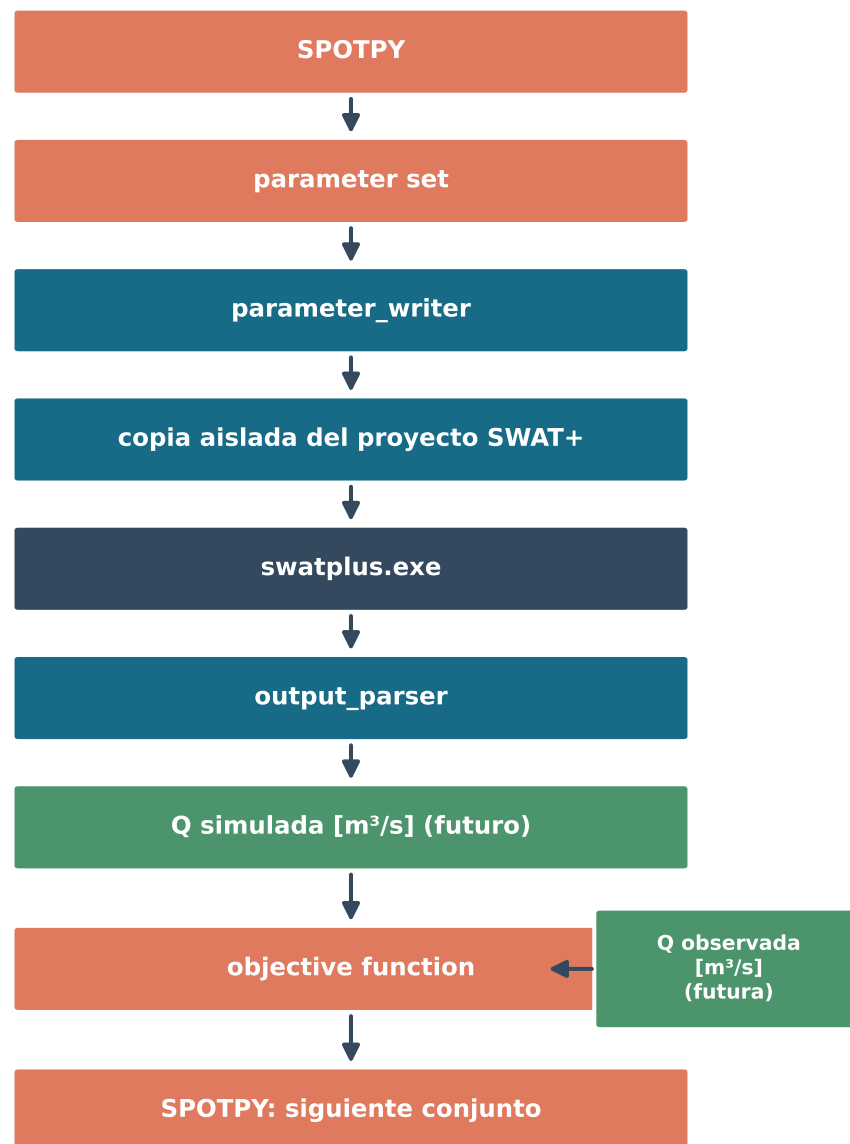
Los tests cubren éxito, fallo y timeout. Una ejecución fallida genera una excepción con causa; no debe contaminar otros workers. SPOTPY sólo necesita que `simulation()` traduzca el conjunto candidato a esa secuencia y devuelva la serie.

La demostración usa Python como ejecutable externo. Sustituirlo por `swatplus.exe` requiere verificar archivos, parámetros, unidades y salida real; no basta cambiar el nombre del comando.

Capítulo 11

Arquitectura futura SWAT+ IGP

SPOTPY + futuro SWAT+ IGP



Arquitectura propuesta: el output real de SWAT+ aún no está implementado.

Figura 11.1: Arquitectura propuesta; no representa una ejecución SWAT+ ya implementada.

Faltan para integración real: ejecutable/version exacta; copia controlada del proyecto; parámetros, archivos y semántica de modificación; archivo/columna de caudal; unidades; estación y calendario; warm-up; reinicio; baseline; manejo de fallos y concurrencia. La Q futura se desea en m^3/s , pero el laboratorio actual no define la unidad de q_{obs} .

Cada worker debe poseer una copia mutable independiente. Compartir un proyecto produce carreras, outputs mezclados y corrupción. Se debe verificar primero una sola corrida secuencial y su reproducibilidad bit a bit o dentro de tolerancia documentada.

Capítulo 12

Capacidades avanzadas evaluadas

12.1. Firmas hidrológicas

Sobre 30 días se calcularon media (4.1473), Q5 (9.585), Q50 (3.1), Q95 (0.6585), coeficiente de variación (0.7409), razón media/mediana (1.3378), autocorrelación lag-1 (0.0804) y 0 % de ceros. Las firmas con dimensión de Q conservan unidad desconocida. No se forzaron frecuencias anualizadas, extremos anuales, recesión ni baseflow.

12.2. NSGA-II

NSGA-II conserva diversidad por dominancia y crowding [2]. Se minimizaron por separado $1 - \text{NSE}$, $1 - \text{KGE}$ y $|\text{PBIAS}|/100$. Se guardaron 48 evaluaciones y 8 filas no dominadas. Un frente de Pareto describe trade-offs; no produce un ganador sin preferencias. La ejecución emitió un warning numérico de crowding por rango objetivo nulo en un subconjunto; terminó y generó evidencia, pero el presupuesto es sólo didáctico. PA-DDS continúa marcado beta en la fuente instalada.

12.3. DREAM y posterior

DREAM evoluciona múltiples cadenas mediante diferencias adaptativas [9]. Con siete cadenas y 140 registros, los $R\text{-hat}$ finales estuvieron entre 2.07 y 4.47, muy por encima del criterio 1.2: **no convergió**. El diagnóstico se relaciona con la comparación entre/ dentro de cadenas [4]. El top 20 % por likelihood es demostrativo y no se presenta como posterior robusta. DE-MCz y Geweke quedan no probados.

12.4. Bases

RAM, CSV y SQLite conservaron 20 registros idénticamente muestreados. RAM es efímera; CSV transparente; SQLite consultable. En Windows, SQL exige `dbname` relativo porque SPOTPY lo reutiliza como nombre de tabla. HDF5 no se probó por ausencia de PyTables.

Capítulo 13

Reproducibilidad

Desde Anaconda Prompt:

```
cd C:\Users\ACER\Documents\GitHub\spotpy_tests
conda activate spotpy-igp
python 01_smoke_test\run_smoke_test.py
python run_experiment.py --example rosenbrock --algorithm mc --runs 100
python 03_hydrology_demo\run_demo.py
python run_experiment.py --example hydrology --algorithm lhs --runs 100
python run_experiment.py --example hydrology --algorithm fast --runs 325
python run_experiment.py --example hydrology --algorithm sceua --runs 500
python run_experiment.py --example hydrology --algorithm dds --runs 500
python 08_external_model_adapter\run_demo.py
python -m pytest -v
```

Los experimentos avanzados se ejecutan mediante los scripts `run_signatures.py`, `run_nsga2.py`, `run_dream.py`, `run_efast.py` y `run_backends.py`, cada uno desde su directorio numerado.

Para recompilar este documento desde la raíz:

```
cd latex
xelatex -interaction=nonstopmode -halt-on-error -jobname=SPOTPY_IGP_Manual main.tex
bibtex SPOTPY_IGP_Manual
xelatex -interaction=nonstopmode -halt-on-error -jobname=SPOTPY_IGP_Manual main.tex
xelatex -interaction=nonstopmode -halt-on-error -jobname=SPOTPY_IGP_Manual main.tex
```

La semilla 42 está fijada donde la API lo permite. Cada ejecución normal crea identificador con microsegundos y no sobrescribe runs anteriores.

Capítulo 14

Conclusiones

El laboratorio prueba que SPOTPY puede separar modelo, datos, objetivos y algoritmos, persistir experimentos y controlar procesos externos. FAST, SCE-UA y DDS ya aportan un camino claro hacia sensibilidad y calibración alrededor de SWAT+.

La principal brecha no es añadir más samplers. Es definir científicamente inputs/units, verificar un proyecto SWAT+, separar calibración/validación, adoptar una likelihood defendible y garantizar directorios por worker. DREAM demostró correctamente una no-convergencia; NSGA-II demostró el contrato multiobjetivo sin declarar un frente estable.

El próximo paso recomendado es una única corrida SWAT+ reproducible sobre copia controlada, con writer/parser testeados y Q observada/simulada realmente expresada en m^3/s . Sólo después corresponde aumentar evaluaciones o paralelizar.

Bibliografía

- [1] R. I. Cukier, C. M. Fortuin, K. E. Shuler, A. G. Petschek, and J. H. Schaibly. Study of the sensitivity of coupled reaction systems to uncertainties in rate coefficients. i. theory. *The Journal of Chemical Physics*, 59(8):3873–3878, 1973. doi: 10.1063/1.1680571.
- [2] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multi-objective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2): 182–197, 2002. doi: 10.1109/4235.996017.
- [3] Qingyun Duan, Soroosh Sorooshian, and Vijai Gupta. Effective and efficient global optimization for conceptual rainfall-runoff models. *Water Resources Research*, 28(4):1015–1031, 1992. doi: 10.1029/91WR02985.
- [4] Andrew Gelman and Donald B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992. doi: 10.1214/ss/1177011136.
- [5] Hoshin V. Gupta, Harald Kling, Koray K. Yilmaz, and Guillermo F. Martinez. Decomposition of the mean squared error and nse performance criteria: Implications for improving hydrological modelling. *Journal of Hydrology*, 377(1–2):80–91, 2009. doi: 10.1016/j.jhydrol.2009.08.003.
- [6] Tobias Houska, Philipp Kraft, Alejandro Chamorro-Chavez, and Lutz Breuer. Spotting model parameters using a ready-made python package. *PLOS ONE*, 10(12):e0145180, 2015. doi: 10.1371/journal.pone.0145180.
- [7] Andrea Saltelli, Stefano Tarantola, and Karen P.-S. Chan. A quantitative model-independent method for global sensitivity analysis of model output. *Technometrics*, 41(1):39–56, 1999. doi: 10.1080/00401706.1999.10485594.
- [8] Bryan A. Tolson and Christine A. Shoemaker. Dynamically dimensioned search algorithm for computationally efficient watershed model calibration. *Water Resources Research*, 43(1): W01413, 2007. doi: 10.1029/2005WR004723.
- [9] Jasper A. Vrugt, Cajo J. F. ter Braak, Cees G. H. Diks, Bruce A. Robinson, James M. Hyman, and Dave Higdon. Accelerating markov chain monte carlo simulation by differential evolution with self-adaptive randomized subspace sampling. *International Journal of Nonlinear Sciences and Numerical Simulation*, 10(3):273–290, 2009. doi: 10.1515/IJNSNS.2009.10.3.273.